AA - Lab 3
Cititi si intelegeti bine reducerile Turing si Halting Problem pentru examen.

REDUCERI TURING

Probleme de decizie

Definitie

f : Σ* → {FALSE, TRUE}
O masina Turing M poate **decide/ accepta** o problema f
M **decide** f ⇔ pentru orice cuvant w avem:
$M[w] \rightarrow TRUE$ cand f(w) = TRUE
$M[w] \rightarrow FALSE$ cand f(w) = FALSE
(cu alte cuvinte: M se opreste pentru orice cuvant și dă răspunsul corect)
M **accepta** f ⇔ pentru orice cuvant w avem:
$M[w] \rightarrow TRUE$ cand f(w) = TRUE
$M[w]$ cicleaza (notat: $M[w] \rightarrow \perp$) sau $M[w] \rightarrow FALSE$ cand f(w) = FALSE
(cu alte cuvinte: M se opreste pentru orice cuvant pentru care f(w) = TRUE)

Proprietati

Problema f decidabila => f acceptabila
Fie M o masina Turing care decide problema f si M* o masina Turing care accepta problema f. Vom construi masina M* folosind M, definind comportamentul ei pentru orice cuvant notat w:

```
M*[w]:      run M[w]            ← se opreste mereu pentru ca M decide f
            if(M[w] →TRUE)
                transition to Y
            else loop           ← un ciclu infinit precum while(true)
```

O problema poate fi acceptata de mai multe masini Turing
Vom construi 2 masini Turing, M si M*, care accepta aceeasi problema.
Construim M* pornind de la M.

```
M*[w]:      run M[w]            ← daca M[w] nu se termina, M*[w] va cicla aici
            muta cursor la stanga apoi la dreapta   ← altfel, M*[w] → FALSE sau M*[w] → TRUE
                (operatia nu face nimic overall,
                dar face ca masinile sa fie diferite
                prin 2 tranzitii in plus)
            return result
```

Deci, pentru orice cuvant w, M* se comporta la fel ca M, iar M accepta problema f
=> M* accepta f

O problema poate fi decisa de mai multe MT
Aceeasi constructie a lui M* ca mai devreme, dar de data aceasta stiim ca *run M[w]* se termina pentru orice cuvant w => si M se va termina pentru orice cuvant w

O masina Turing nu poate accepta/decide 2 probleme DIFERITE

Complementul unei probleme

Definim **complementul unei probleme f** (notat $\overline{f}$)

$\overline{f}$(w) = TRUE daca f(w) = FALSE
FALSE daca f(w) = TRUE

f decidabila => $\overline{f}$ decidabila

Fie M o masina Turing care decide f.

Construim masina Turing M* pornind de la M astfel incat M* sa decida $\overline{f}$.

$M_{\overline{f}}$[w]: *if(M_f[w] →TRUE)*
transition to N
else
transition to Y

f acceptabila =X> $\overline{f}$ acceptabila !!!! Atentie la =X> care inseamna NU rezulta

f si $\overline{f}$ acceptabile => f si $\overline{f}$ decidabile

Fie masinile Turing M_f si $M_{\overline{f}}$ care accepta problemele f si $\overline{f}$.

Demonstram ca M decide f. Similar si M* se poate arata ca decide $\overline{f}$**.**

M[w]: *while(1) {* ← *Atentie la acest exemplu, posibil sa se dea la examen*
step(M_f[w]);
step($M_{\overline{f}}$[w]);
if(M_f[w] accepta)
transition to Y
if($M_{\overline{f}}$[w] accepta)
transition to N
}

HALTING PROBLEM & LOOPING PROBLEM

HALTING PROBLEM

f_h(enc(M, w)) = TRUE daca M[w] se termina/halts
FALSE altfel

Obs: enc(M) = encodarea masinii Turing M (cum am scrie masina ca input pe banda)
Ce tip de problema este Halting Problem?

- HP: acceptabila, NEdecidabila

LOOPING PROBLEM

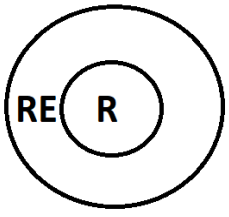
- complementul lui Halting Problem
- LP: NEacceptabila, NEdecidabila

f_l(M, w) = TRUE daca M[w] cicleaza
FALSE altfel

OBS!!!!:
Halting Problem si Looping Problem primesc atat o masina Turing cat si un cuvant ca input

Clase de probleme

R = multimea problemelor **decidabile**
= {f in $\mathbb{D}$| f este decisa de o masina Turing}



RE = multimea problemelor **acceptabile**
= {f in $\mathbb{D}$) | f este acceptata de o masina Turing}
R ⊂ RE

Masina Turing Universala

- primeste ca input o pereche (M, w)
- ruleaza pas cu pas M pe w

Observatie:
→ Masina Turing M se poate scrie pe banda sub forma enc(M) => multimea masinilor Turing este infinit numarabila (ca multimea numerelor naturale/intregi)
→ problemele de decizie sunt o multime infint nenumarabila (ca multimea numerelor reale)
DECI, avem infinit mai multe probleme de decizie decat avem masini Turing care sa le rezolve.

REDUCERI TURING

Notatia **$f \leq_m g$**
→ g este "mai grea/dificila" decat f / f este "mai usoara"
→ f se reduce Turing la g
→ daca avem un algoritm/masina Turing pentu g, avem una si pentru f
$f \leq_m g \Leftrightarrow$ exista o transformare computabilă t (se poate calcula cu o masina Turing ca in laborator 1) astfel incat, pentru orice cuvant w pentru care f(w) = TRUE, avem g(t(w)) = TRUE

Ceea ce trebuie sa demonstam: **f(w) = TRUE ⇔ g(T(w)) = TRUE**
1. construim M_t
2. Aratam ca f(w) = TRUE => g(t(w)) = TRUE
3. Aratam ca g(t(w)) = TRUE => f(w) = TRUE

De retinut!
- **$f \notin R \Rightarrow g \notin R$**
- **$f \notin RE \Rightarrow g \notin RE$**
- **$g \in R \Rightarrow f \in R$**
- **$g \in RE \Rightarrow f \in RE$**

Proprietati:

1. Pentru orice probleme f si g pentru care avem **$f \leq_m g$** si **$g \leq_m f$** => **$g \equiv_m f$** (echivalente Turing/"la fel de grele")
2. Pentru orice problema f avem **$f \leq_m f$**
3. Pentru orice probleme f, g, h pentru care **$f \leq_m g$, $g \leq_m h$** => **$f \leq_m h$**
4. Pentru orice probleme f si g pentru care avem **$f \leq_m g$ =X> $g \leq_m f$!!!!!**

Probleme tipice:
- **aratati ca f este o problema decidabila($\in$R) sau acceptabila($\in$RE)**
  → construim o Masina Turing care accepta / decide problema resectiva
- **aratati ca f este o problema NEdecidabila($\notin$R)/ NEacceptabila($\notin$RE)**
  → folosim REDUCERI TURING

Exemple

1)

Aratati ca f ∈ RE

f(M_1, M_2) = TRUE daca exista w astfel incat M_1[w] si M_2[w] accepta
FALSE altfel

NU ASA!!

```
M(M1, M2):
   inputs = ∑*
   for w in inputs
      if(M1[w] si M2[w] acepta)
         transition to Y
   transition to N
```

De ce nu e bine?
Ceea ce face M e sa ruleze mașinile pe orice cuvant. Dar putem la primul pas sa avem M_1 care se opreste si M_2 cicleaza pe primul cuvant incercat, si nu vom mai ajunge vreodata la urmatoarea iteratie.

run M_1(0) ← daca cicleaza aici,
run M_1(1) ← nu se mai ajunge aici
.....

Deci, la fiecare pas, trebuie sa verificam doar rularea unui <u>numar finit de pasi</u> pe un <u>numar finit de cuvinte</u>.

CORECT:

```
M(M1, M2):
   k = 1;
   input = [0];
   while(true) {
      for w in input  {  // multime finita
         if(M1[w] accepta in maxim k pasi
         si M2[w] accepta in maxim k pasi)
            return 1
         input.append(k)
         k++
      }
   }
```

Step(w) niciodata nu va cicla. Masina M va returna 1 daca se gaseste un cuvant pentru care M_1 si M_2 se termina amandoua in maxim k pasi, altfel masina M construita va cicla. Deci, deoarece M accepta f si conform definitiei lui RE, f este acceptabila.

2)

In ce clasa de probleme sunt:

a) *f_1(M) = TRUE daca M are mai mult de 5 stari*
FALSE altfel
Construim M_{f1}(M): lookup in enc(M) si vedem daca exista macar 5 stari in K
Deci, problema f_1 este decidabila (R). Dar **R** ⊂ **RE**, deci f_1 este si acceptabila (RE)

b) *f_2(w) = TRUE, w e rprezentarea binară a unui nr prim*
FALSE, altfel
Construim o masina Turing (algoritm) care o decide
=> f_2 este decidabila (R) => acceptabila (RE)

3)

Calculati $\overline{f}$: *f(M_1, M_2) = TRUE daca ambele M_1, M_2 acepta w, pentru ORICE cuvant w*
FALSE altfel

$\overline{f}$(M_1, M_2) = TRUE daca (M_1 accepta w si M_2 nu accepta) sau (M_2 accepta si M_1 nu accepta w)
sau (M_1 nu accepta w si M_2 nu accepta)
FALSE altfel

Exercitii cu reduceri Turing:

Practic, demonstratia urmeaza logica urmatoare:
g(input') = 1 => t => f(input) =1
f(input) = 1 => t => g(input') =1

1)
Aratati ca $f_{111} \notin R$
$f_{111}(M^*) = TRUE \Leftrightarrow M^*[111] \rightarrow TRUE$

Rezolvare:
Vom arata ca $f_h \leq_m f_{111}$ unde f_h este Halting Problem.
Gasim o transformare t care primeste inputul lui Halting Problem si intoarce un input pentru f_{111}
M, w —t⟶ M*
Avem de demonstrat ca $f_h(M, w) = TRUE \Leftrightarrow f_{111}(M^*) = TRUE$
Transformarea t:
M*(x): run M[w] ← *Pentru orice cuvant x, M* va rula M(w)*
return x == 111
"=>"
$f_h(M, w) = TRUE$ => M[w] halts => vom returna x == 111
=> cand x este 111, M* va tranziționa în Y=> $M^*[111] \rightarrow TRUE$ => $f_{111}(M^*) = TRUE$
"<="
$f_{111}(M^*) = TRUE$ => $M^*[111] \rightarrow TRUE$ => am ajuns la return
=> run M[w] din M* nu a ciclat => M[w] s-a oprit => $f_h(M, w) = TRUE$
Din ambele directii ale demonstratiei => $f_h \leq_m f_{111}$

2)
$f_p(M^*) = TRUE \Leftrightarrow \forall w$, M* decide dacă w e palindrom
Aratati ca $f_{rev} \notin R$

Rezolvare:
Aratam ca $f_h \leq_m f_p$
Avem de demonstrat ca $f_h(M,w) = TRUE \Leftrightarrow f_p(M^*) = TRUE$
M, w — t⟶ M*
t: **M*(x): run M[w]** // calculeaza rev(x) doar daca M[w] se opreste
check if x is palindrome // operatia de verificare de palindrom e decidabilă
"=>"
$f_h(M, w) = TRUE$ => M[w] halts
=> Din transformarea T, M*[x] decide dacă x e palindrom pentru orice xdeoarece ajunge la return
=> $f_p(M^*) = TRUE$
"=>"
$f_p(M^*) = TRUE$ => M*[x] decide dacă x e palindrom pentru orice x
=> M[w] s-a oprit => $f_h(M, w) = TRUE$
Din ambele directii ale demonstratiei => $f_h \leq_m f_{pev}$

3)
$f_{rev}(M^*) = TRUE \Leftrightarrow \forall w$, M* calculeaza inversul lui w
Aratati ca $f_{rev} \notin R$

Rezolvare:
Aratam ca $f_h \leq_m f_{rev}$
Avem de demonstrat ca $f_h(M,w) = TRUE \Leftrightarrow f_{rev}(M^*) = TRUE$
$M, w \text{ — } t \longrightarrow M^*$

t: **M*(x): run M[w]** // calculeaza rev(x) doar daca M[w] se opreste
return rev(x) // operatia de reverse este computabilă

"=>"

$f_h(M, w) = TRUE \Rightarrow M[w]$ halts
=> Din transformarea T, M*[x] calculeaza rev(x) pentru orice x deoarece ajunge la return
$\Rightarrow f_{rev}(M^*) = TRUE$

"=>"

$f_{rev}(M^*) = TRUE \Rightarrow$ M*[x] calculeaza rev(x) orice x
=> M[w] s-a oprit => $f_h(M, w) = TRUE$

Din ambele directii ale demonstratiei => $f_h \leq_m f_{rev}$

4)
$f_{own}(M^*) = TRUE \Leftrightarrow$ M*[enc(M*)] halts
Aratati ca $f_{own} \notin R$

Rezolvare:
Aratam ca $f_h \leq_m f_{own}$
Avem de demonstrat ca $f_h(M,w) = TRUE \Leftrightarrow f_{own}(M^*) = TRUE$
$M, w \text{ — } t \longrightarrow M^*$

t: **M*(x): if x == enc(M*)**:
run M[w]
else:
halt

"=>"

$f_h(M, w) = TRUE \Rightarrow M[w]$ halts
=> M*[enc(M)] halts
$\Rightarrow f_{own}(M^*) = TRUE$

"=>"

$f_{own}(M^*) = TRUE \Rightarrow$ M*[enc(M*)] halts
=> M[w] halts => $f_h(M, w) = TRUE$

Din ambele directii ale demonstratiei => $f_h \leq_m f_{own}$

5)
$f_{finite}(M^*) = TRUE \Leftrightarrow$ M* se opreste pentru un număr finit de cuvinte
Aratati ca este nedecidabila($\notin R$)

Rezolvare:
$f_h \leq_m f_{finite}$
Avem de demonstrat ca $f_h(M,w) = TRUE \Leftrightarrow f_{finite}(M^*) = TRUE$
$M, w \text{ —} t \text{—} \longrightarrow M^*$
Transformarea t:

M*[x]: run M[w] pentru maxim |x| pași
dacă M[w] s-a oprit:
loop
else:
halt

"=>"

f(M, w) = TRUE => Din transformarea t, M[w] halts în n pași=> M*[w] se oprește pentru orice x cu |x| < n (un nr finit), ciclează pentru orice x, cu |x| >= n

=> M* se opreste pentru un numar finit de cuvinte => $f_{finite}(M^*)$ = TRUE

"<="

$f_{finite}(M^*)$ = TRUE => M* se opreste pentru un numar finit de cuvinte

=> Din transformarea t putem deduce ca M[w] s-a oprit după un nr de pași => $f_h(M, w)$ = TRUE

6)

$f_{set}(A, M^*) = 1 \Leftrightarrow \forall w \in A$, M*[w] halts

Aratati ca este nedecidabila($\notin R$)

Rezolvare:

$f_h \leq_m f_{set}$

Avem de demonstrat ca $f_h(M, w)$ = TRUE $\Leftrightarrow$ $f_{set}(M^*)$ = TRUE

M, w —t—→ A, M*

Transformarea T: Construim A și M care se opreste doar daca x ∈ A*

A = {101}

M*[x]: if x ∈ A

run M[w]

else

loop

"=>"

$f_h(M, w)$ = TRUE => M[w] halts

cum M* se poate opri doar pentru x = 101 => $f_{set}(A, M^*)$ = TRUE

"<="

$f_{set}(A, M^*)$ = TRUE => M* se va opri pentru orice x ∈ A, adica pentru x = 101

=> M[w] halts => $f_h(M, w)$ = TRUE

7)

$f_x(M^*)$ = TRUE $\Leftrightarrow \exists w$, M*[w] scrie un x pe bandă la un moment dat

Rezolvare: $f_h \leq_m f_x$

Avem de demonstrat ca $f_h(M,w)$ = TRUE $\Leftrightarrow$ $f_x(M^*)$ = TRUE

M, w —t—→ M*

Transformarea t: M trebuie sa scrie x pe banda doar daca M[w] se opreste*

M*[y]: run M[w]

// daca M ar avea tranzitii definite pe x/printeaza x, inlocuieste-l cu alta litera

print x

"=>"

$f_h(M,w)$ = TRUE => M[w] halts => M* printeaza x la final => $f_x(M^*)$ = TRUE

"<="

$f_x(M^*)$ = TRUE => M* ajunge sa printeze x, din tansformare

=> il printeaza la final dupa ce a rulat M[w] => M[w] halts => $f_h(M, w)$ = TRUE

8)

$f_{eq}(M_1,M_2)$ = TRUE $\Leftrightarrow \forall w$, $M_1[w] \equiv M_2[w]$; i.e. mașinile au același rezultat (0, 1, ciclare) pentru orice cuvânt.

Rezolvare: $f_h \leq_m f_{eq}$
Avem de demonstrat ca $f_h(M, w) = TRUE \Leftrightarrow f_{eq}(M_1, M_2) = TRUE$
$M,w \text{—t—→} M_1, M_2$
Scoatem M_1 si M_2 din transformare cu acelasi comportament doar daca M[w] se opreste.
Transformarea T:

M_1[x]: transition to Y
***M_2(x):** (acelasi comportament cu M_1[x] doar daca M[w] halts)*
***run M[w]** // w fixat din Halting Problem*
transition to Y

"=>"

$f_h(M, w)$ =TRUE => M[w] halts => M_2[x] acceptă orice x,
iar M_1[x] deja acceptă orice x din constructie
=> $M_1[x] \equiv M_2[x]$ pentru oricare x => $f_{eq}(M_1, M_2) = TRUE$

"<="

$f_{eq}(M_1, M_2) = TRUE \Rightarrow M_1[x] \equiv M_2[x]$ pentru orice x
=> stiind ca M_1[x] din constructie racceptă orice => M_2[x] accepta orice => M_2 mereu ruleaza cu succes M[w] => M[w] se opreste => $f_h(M, w) = TRUE$

9)
$f_{all}(M) = TRUE \Leftrightarrow \forall w$, M[w] halts
$f_{eq}(M_1, M_2)$=TRUE $\Leftrightarrow \forall w, M_1[w] \equiv M_2[w]$

Rezolvare: $f_{all} \leq_m f_{eq}$
$M \text{—t—→} M_1, M_2$
Construim t, astfel incat sa obtinem 2 masini cu acelasi comportament doar daca M[w] se opreste pentru orice w
Transformarea t: // atentie la diferenta de nuanta fata de exercitiul anterior

M1[x]: transition to Y
***M2[x]: run M[x]** // diferenta este aici*
transition to Y

"=>"

$f_{all}(M) = TRUE$ => pentru orice x, M[x] halts => M_2[x] se accepta orice x
=> Cum M_1[x] din constructie acceptă
M_1[x] si M_2[x] vor accepta amândouă orice x
=> $M_1[x] \equiv M_2[x]$ oricare x => $f_{eq}(M_1, M_2) = TRUE$

"<="

$f_{eq}(M_1, M_2) = TRUE \Rightarrow M_1[x] \equiv M_2[x]$ orice x => cum am construit M_1[x] ca sa accepte
=> M_2[x] si ea acceptă orice x => M[x] acceptă orice x => $f_{all}(M)$=TRUE

10)
Aratati ca $f_{any} \leq_m f_{eq}$
$f_{eq}(M_1, M_2) = TRUE \Leftrightarrow \forall w, M_1[w] \equiv M_2[w]$ (au acelasi rezultat sau ambele cicleaza)
$f_{any}(M) = TRUE \Leftrightarrow \exists w$, M[w] halts

Rezolvare:
$M \text{—t—→} M_1, M_2$
Transformarea t:

M1[x]: transition to Y
M2[x]:
k = 1

```
inputs = [0]
while true {
        for w in inputs
                step(M[w]) // rulam o singura tranzitie, nu toata M[w]
                if(M[w] s-a oprit)
                        transition to Y
        k++
        inputs.append(k)
}
```

"=>"

$f_{any}(M)$ = TRUE => exista w a.i M[w] se opreste

=> $M_2[x]$ se acceptă

la fel cum face din constructie si $M_1[x]$ pentru orice x

=> $M_1[x] \equiv M_2[x]$ pentru orice x => $f_{eq}(M_1,M_2)$ = TRUE

"<="

$f_{eq}(M_1, M_2)$ = TRUE => $M_1[x]$ si $M_2[x]$ au acelasi comportament pentru orice x

=> Cum M_1 accepta mereu, inseamna ca M_2 ajunge sa gaseasca un cuvant w pentru care M[w] se opreste => exista un w pentru care M[w] se opreste => $f_{any}(M)$ = TRUE

Deci $f_{any} \leq_m f_{eq}$

11)

$f_p(M)$ = TRUE $\Leftrightarrow \forall w$, M decide dacă w e palidnrom

$f_{eq}(M_1, M_2)$=TRUE $\Leftrightarrow \forall w$, $M_1[w] \equiv M_2[w]$

Rezolvare: $f_p \leq_m f_{eq}$

$M \text{—t—→} M_1, M_2$

Construim t, astfel incat sa obtinem 2 masini cu acelasi comportament doar daca M decide dacă w e palindrom

Transformarea t:

```
M1[x]: transition to Y
M2[x]: check if x is palindrome
        run M[x]
        if M correctly decided if x is palindrome:
        transition to Y
        else:
            transition to N
```

"=>"

$f_p(M)$ = TRUE => pentru orice x, M decide dacă e palindrom (deci se termină) => $M_2[x]$ accepta orice x

=> Cum $M_1[x]$ din constructie acceptă

$M_1[x]$ si $M_2[x]$ vor accepta amândouă orice x

=> $M_1[x] \equiv M_2[x]$ oricare x => $f_{eq}(M_1, M_2)$ = TRUE

"<="

$f_{eq}(M_1, M_2)$ = TRUE => $M_1[x] \equiv M_2[x]$ orice x => cum am construit $M_1[x]$ ca sa accepte

=> $M_2[x]$ si ea acceptă orice x => M decide corect daca orice x e palindrom (deci se și termină mereu) => $f_p(M)$=TRUE

12)
$f_{own}(M) = TRUE \Leftrightarrow M[enc(M)]$ halts
$f_{eq}(M_1, M_2)=TRUE \Leftrightarrow \forall w, M_1[w] \equiv M_2[w]$

Rezolvare: $f_{own} \leq_m f_{eq}$
$M \text{—t—→} M_1, M_2$
Construim t, astfel incat sa obtinem 2 masini cu acelasi comportament doar daca M[enc(M)] halts
Transformarea t:

M1[x]: transition to Y
M2[x]: if x == enc(M):
run M[x]
transition to Y

"=>"
$f_{own}(M) = TRUE$ => M[enc(M)] halts => $M_2[x]$ accepta orice x
=> Cum $M_1[x]$ din constructie acceptă
$M_1[x]$ si $M_2[x]$ vor accepta amândouă orice x
=> $M_1[x] \equiv M_2[x]$ oricare x => $f_{eq}(M_1, M_2) = TRUE$
"<="
$f_{eq}(M_1, M_2) = TRUE$ => $M_1[x] \equiv M_2[x]$ orice x => cum am construit $M_1[x]$ ca sa accepte
=> $M_2[x]$ si ea acceptă orice x => M[enc(M)] halts => $f_{own}(M)=TRUE$

13)
Fie o problemă de decizie f; luăm transformarea t: $\Sigma^* \rightarrow \Sigma^*$, t(w) = w
$f(w) = TRUE \Leftrightarrow f(t(w)) = TRUE$
deci $f \leq_m f$

Fie problemele de decizie f, g, h cu:
$f \leq_m g$ (deci există transformarea t_1 a.î. $f(w) = TRUE \Leftrightarrow g(t_1(w)) = TRUE$)
$g \leq_m h$ (deci există transformarea t_2 a.î. $g(w) = TRUE \Leftrightarrow h(t_2(w)) = TRUE$)

luăm transformarea $t = t_1 \circ t_2$
$f(w) = TRUE \Leftrightarrow h(t_2(t_1(w))) = TRUE$
$f(w) = TRUE \Leftrightarrow h(t(w)) = TRUE$
din exercițiul 6 din laboratorul 3, știm că compunerea a două funcții computabile e computabilă, deci t e computabilă, deci
$f \leq_m h$